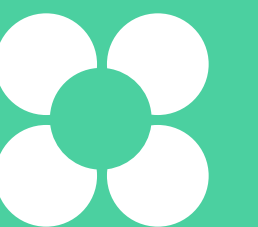


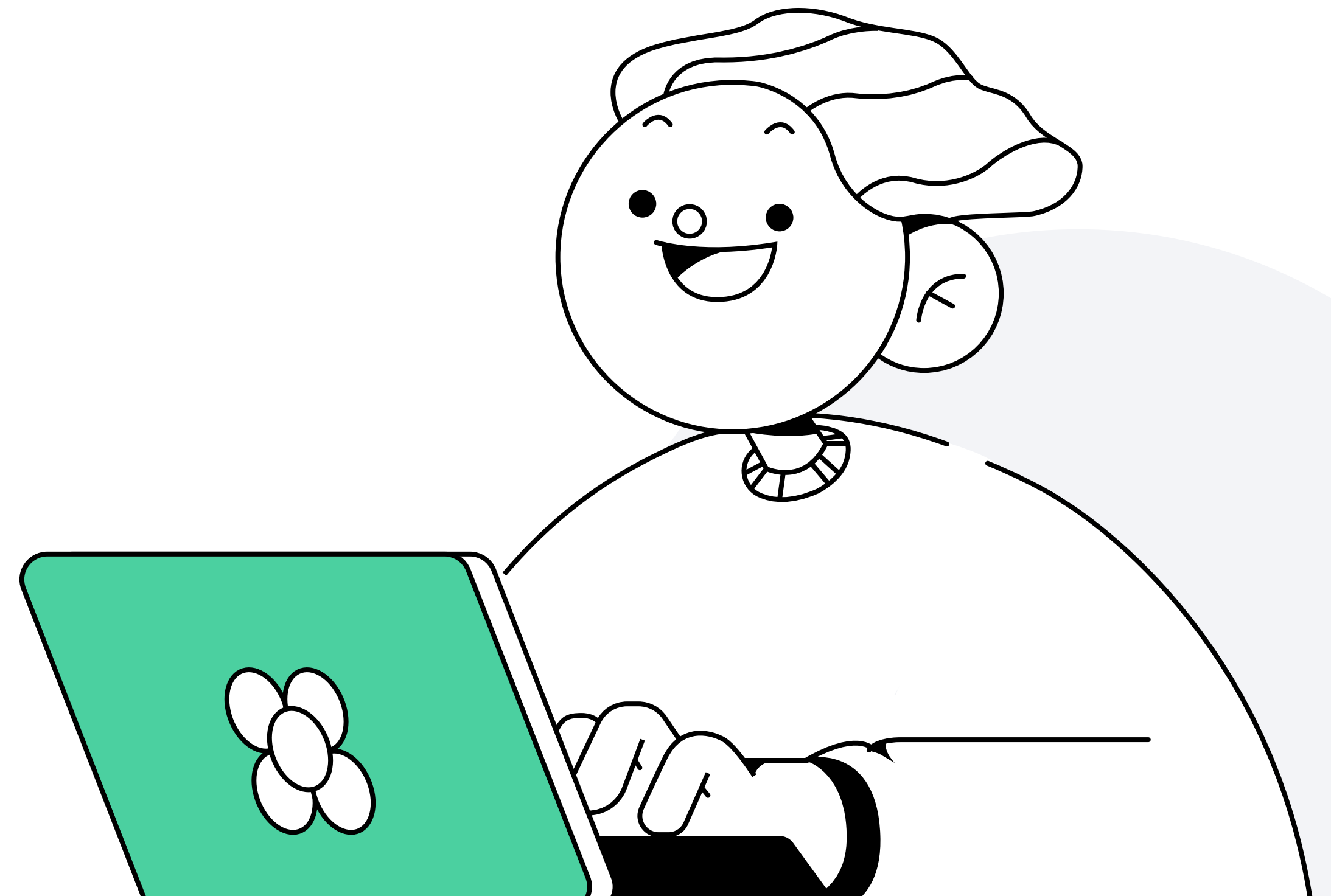
Лямбда-выражения и функциональные интерфейсы

Денис Коротков
Software Engineer, Harman Connected Service

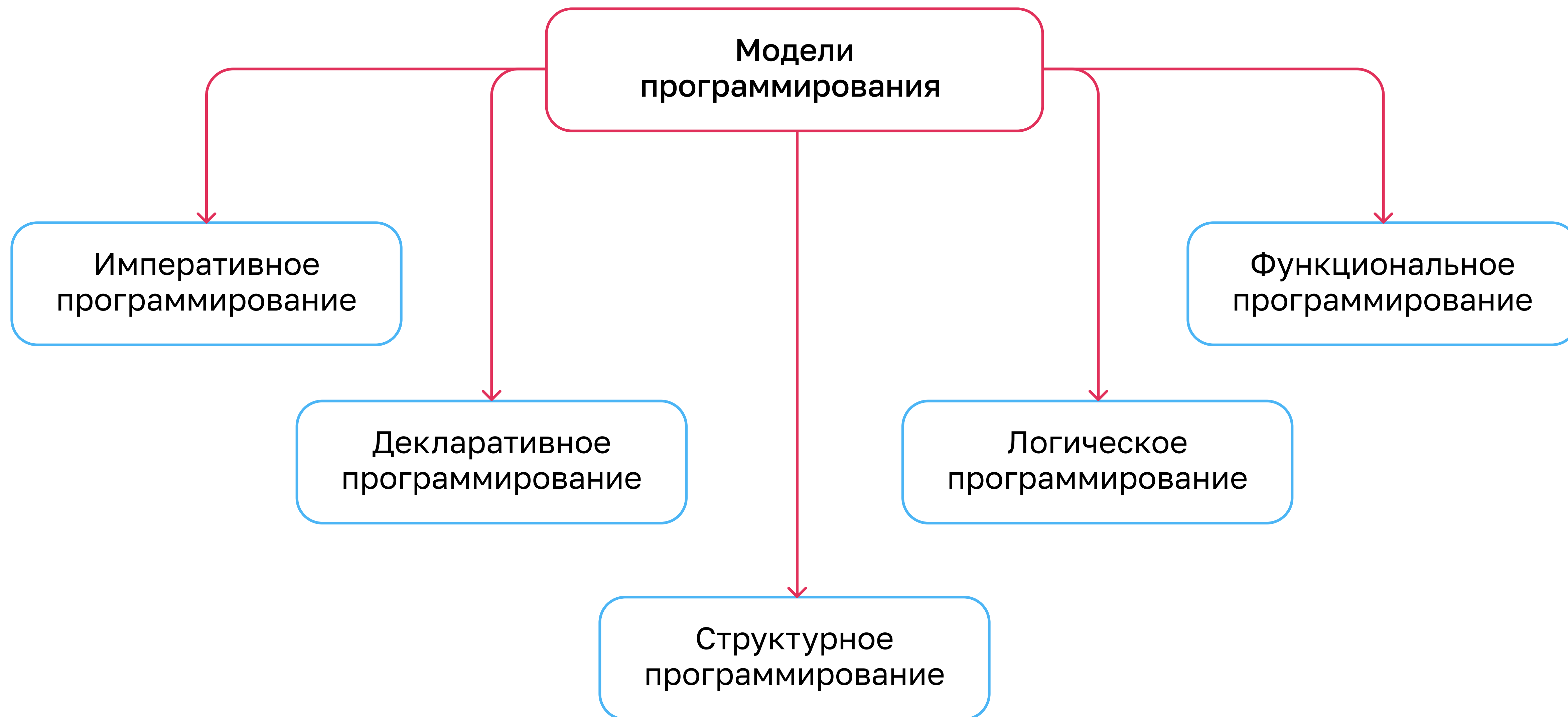


Цели занятия

- ① Разобраться, что такое:
 - лямбда-выражения
 - функциональные интерфейсы
 - ссылки на метод
- ② Понять, как с их помощью улучшать и упрощать код



Функциональное программирование



Императивное программирование

- В исходном коде программы записываются инструкции (команды)
- Инструкции должны выполняться последовательно
- Данные, получаемые при выполнении предыдущих инструкций, могут читаться из памяти последующими инструкциями
- Данные, полученные при выполнении инструкции, могут записываться в память

Декларативное программирование

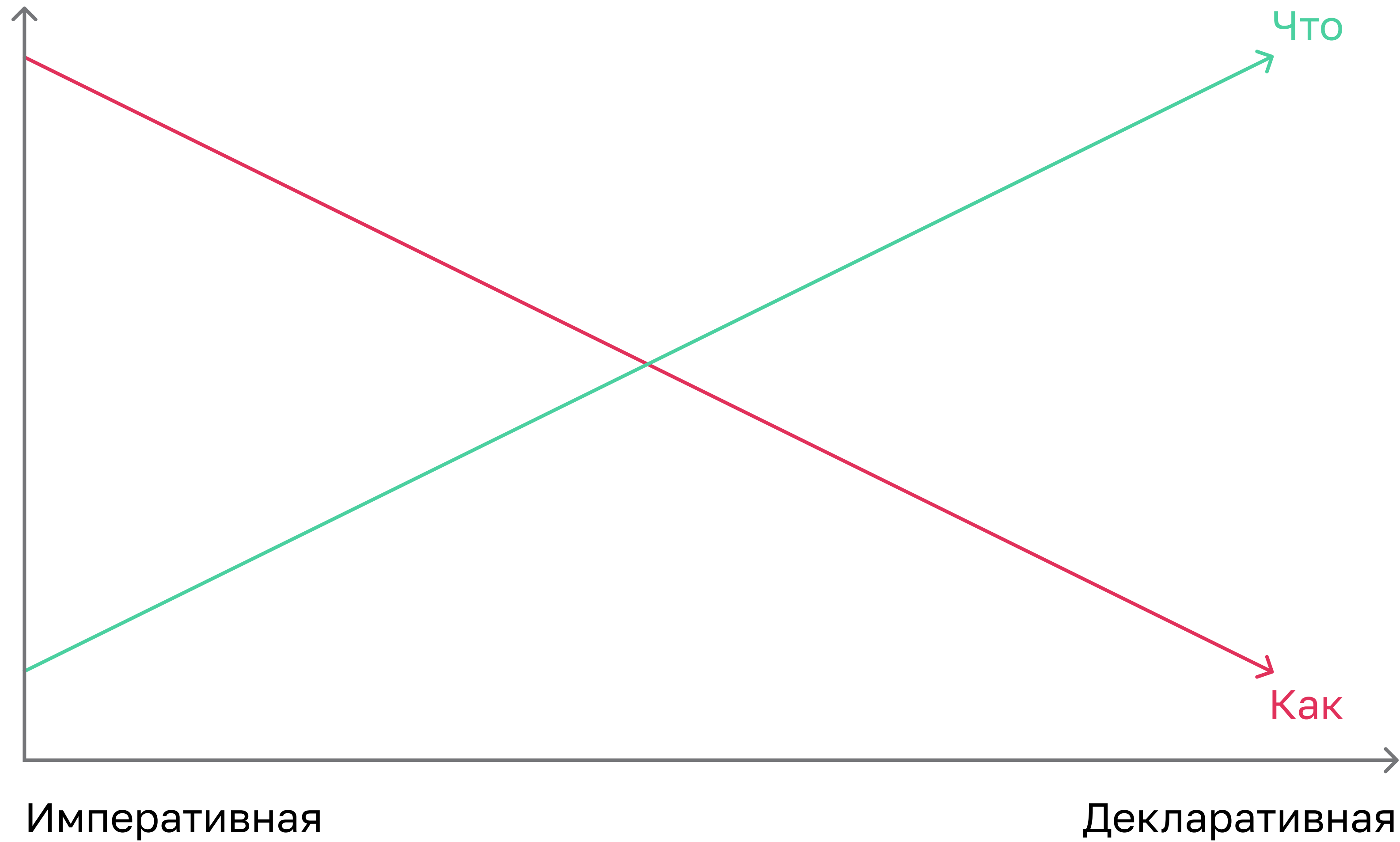
Парадигма программирования, в которой задаётся спецификация решения задачи, то есть описывается проблема и ожидаемый результат



Декларативное программирование

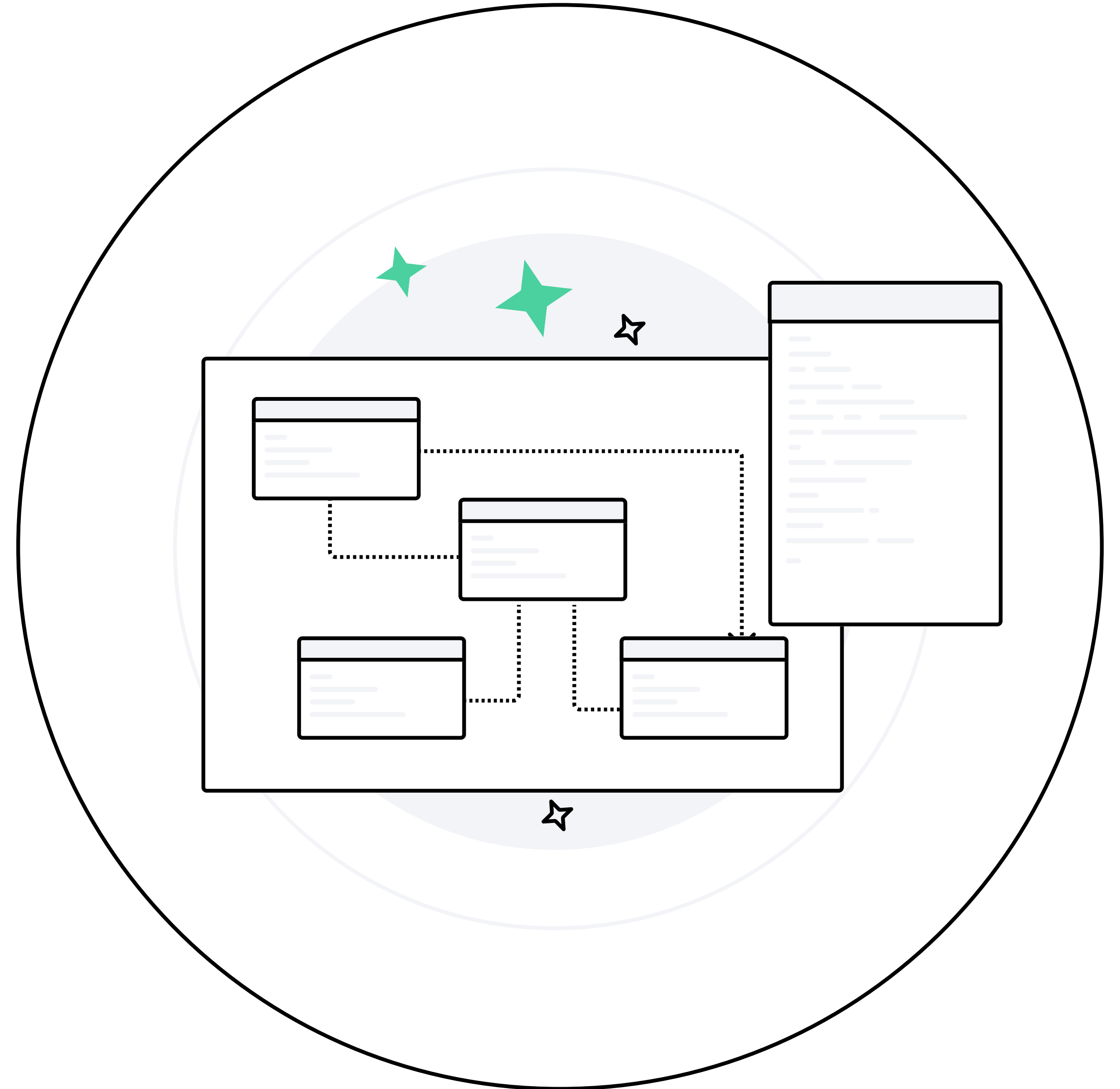
```
SELECT ProductId, ProductName
FROM (SELECT ProductId, ProductName
      FROM Goods
      WHERE Category = (SELECT CategoryId
                        FROM Categories
                        WHERE CategoryName = 'Комплектующие ПК'))
      ) AS Query;
```


Модели программирования



Структурное программирование

Парадигма программирования, в основе которой лежит представление программы в виде иерархической структуры блоков

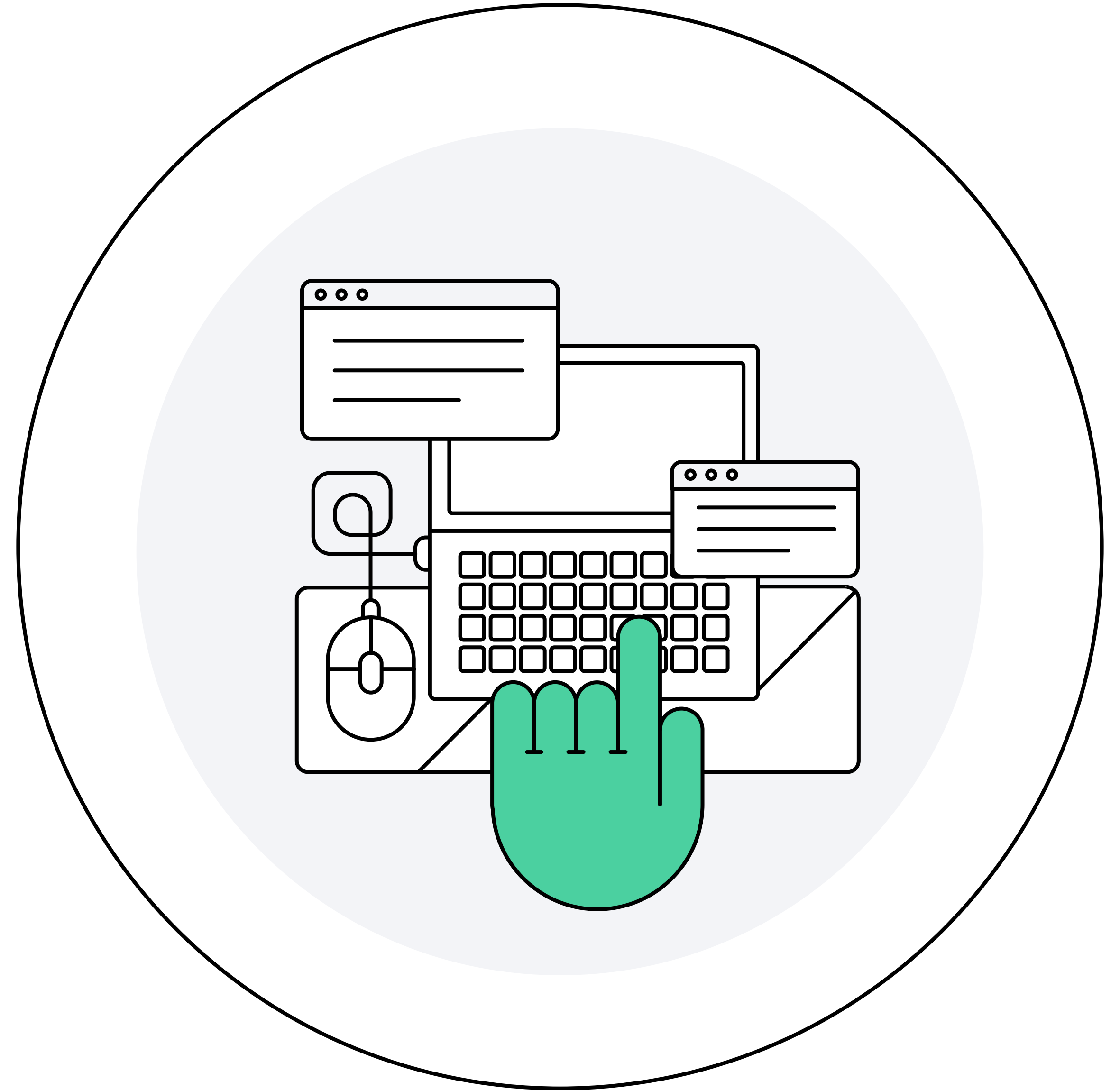


Как выделить инвестирование

```
# Lang "fb"
Dim As String n,s,t,n2,t2
Cls : Print "Добро пожаловать!" 'Заголовок в первой строке
Do
    'Цикл, выводящий линию под заголовком, на всю ширину экрана
    For i As Integer=1 To Loword(Width)
        Print "=";
    Next i
    'Ввод символьных данных от пользователя
    Input "Имя: ",n
    Input "Фамилия: ",s
    Input "Отчество: ",t
    'Вырезаем копию первых символов из имени и отчества
    n2=Left(n,1)
    t2=Left(t,1)
    'Выводим результат
    Print "Ваше имя кратко: "; s; " "; n2; ". "; t2; "."
    Print "Повторить программу? (y/n) "
    Loop While LCase(Input(1))="y"
End 0
```


Логическое программирование

Парадигма программирования, основанная на автоматическом доказательстве теорем, а также раздел дискретной математики, изучающий принципы логического вывода информации на основе заданных фактов и правил вывода



Логическое программирование

```
parent("Tom","Jake").  
parent("Janna","Jake").  
parent("Tom","Tim").  
male("Tom").  
male("Tim").  
male("Jake").  
female("Janna").  
  
brother(X,Y):-  
parent(Z,X),parent(Z,Y),male(X),male(Y),X\=Y.
```


Класс и методы функционального программирования



Класс и методы функционального программирования

```
public class MyClass {  
    int param;  
    String name;
```

```
    public int someMethod1(int diff) {  
        return param + diff;  
    }
```

```
    public int someMethod2(int diff) {  
        param += diff;  
        return param;  
    }  
}
```

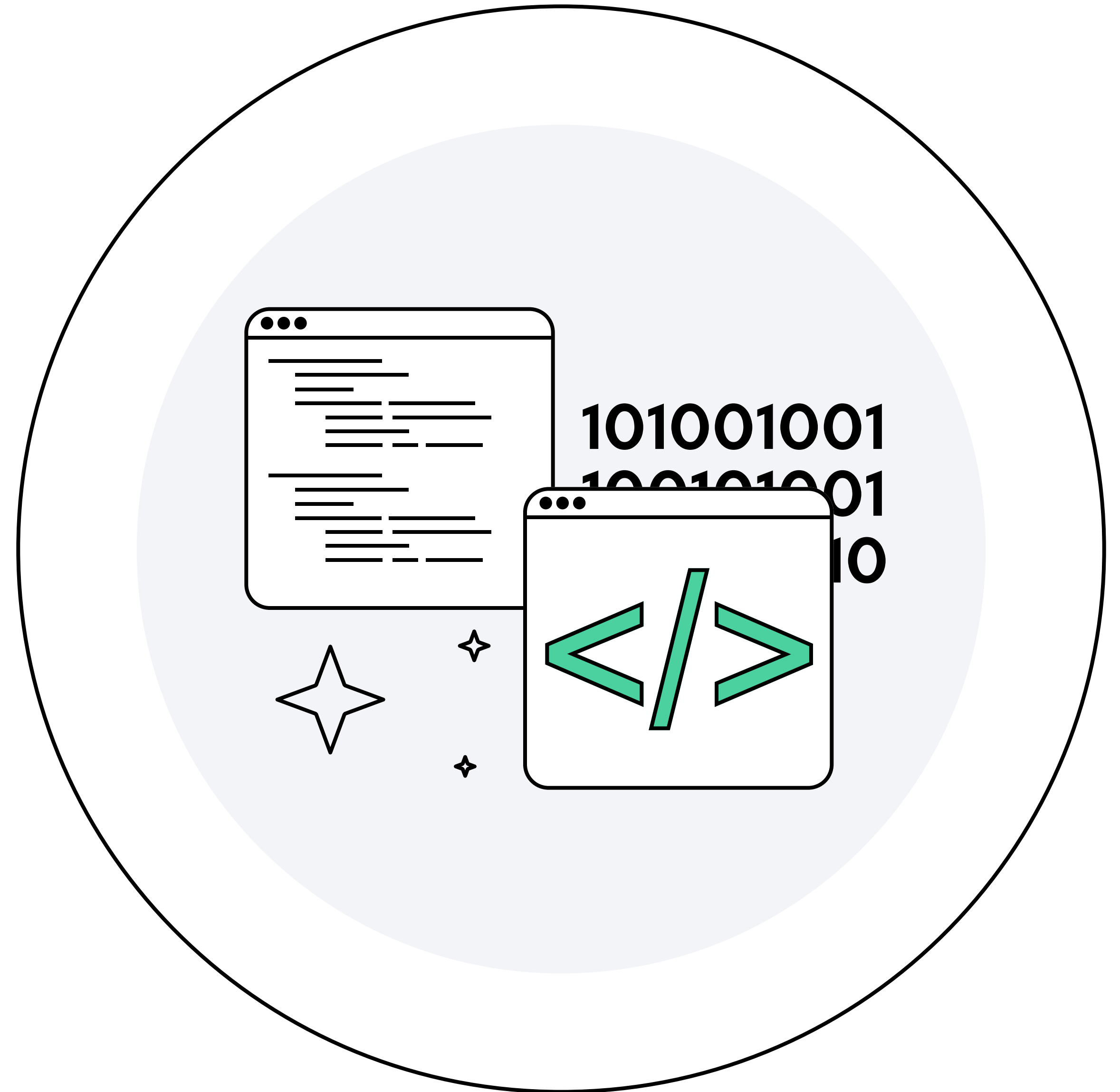
Побочные эффекты (side effects)

- Изменение объекта (его состояния)
- Операции ввода-вывода
- Изменение значений глобальных переменных
- Обработка исключительных ситуаций
- Изменение поведения процессора при работе с числами
- Вызов метода, который совершает любое из вышеперечисленного

Детерминированные функции

Функции, результат работы которых зависит только от входных параметров и ни от чего больше: ни от времени, ни от количества вызовов, ни от чего-либо ещё.

Например, сколько бы раз вы ни спросили у Google про число Пи, его ответ не изменится



Детерминированность и побочный эффект

Методы с побочными эффектами часто могут быть недетерминированы:

```
public class MyClass {  
    int param;  
    String name;  
  
    public int someMethod1(int diff) {  
        return param + diff;  
    }  
  
    public int someMethod2(int diff) {  
        param += diff;  
        return param;  
    }  
}
```

Чистая функция

- Детерминирована
- Не обладает побочными эффектами

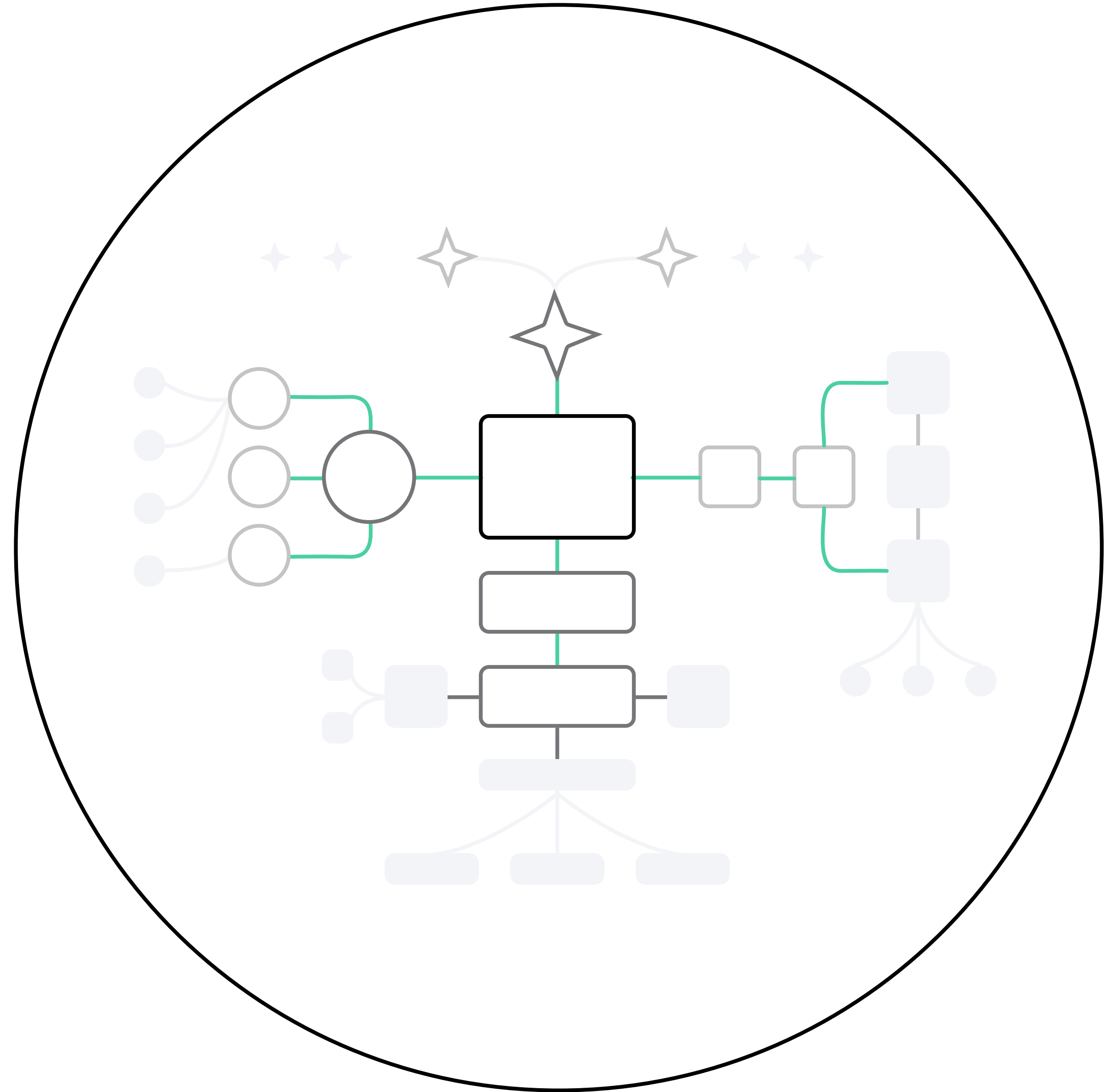


Чистая функция

Чтобы отделить чистые
функции от «грязных», иногда
используют монады

Монада

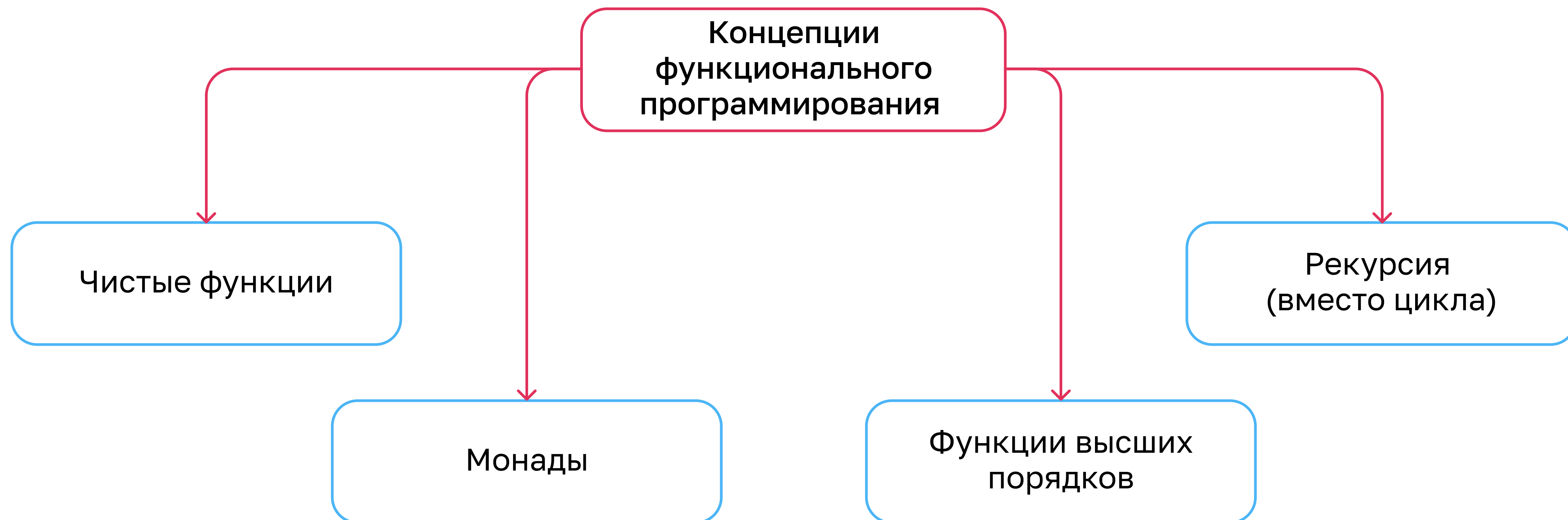
Монада – абстракция линейной цепочки связанных вычислений, то есть нечто такое, что объединяет несколько вычислений в определённом порядке



Функции высших порядков

Функции высших порядков — это функции, которые могут принимать в качестве аргументов другие функции и в качестве результата также возвращать функцию





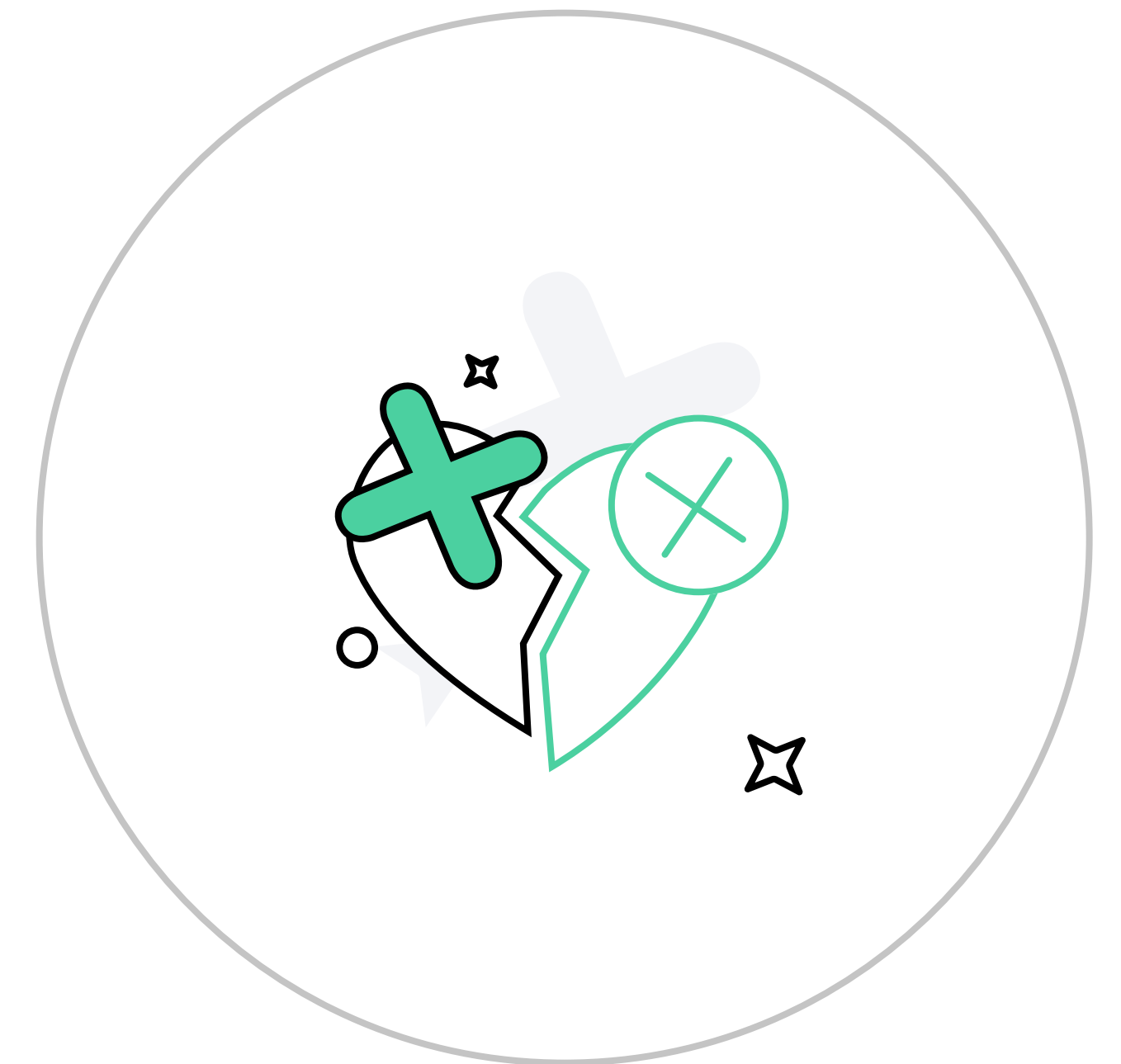
Польза функционального программирования

- ⊕ Лёгкое распараллеливание кода, так как нет side effects
- ⊕ Возможность «сэкономить», когда много раз вызывается функция с теми же аргументами
- ⊕ Лёгкое тестирование



Отрицательные стороны функционального подхода

- Некоторые места в программе просто нереально сделать без побочных эффектов, так как любая программа как минимум совершает операции ввода-вывода
- Из-за «нехранимости» состояния, при вычислении функций постоянно выделяется память под вычисления, и она же собирается сборщиком мусора, когда больше не нужна

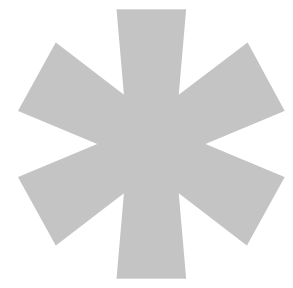


Функция в математике и программировании

- Функция в программировании (метод) – фрагмент кода, который можно вызвать из другого места программы для его выполнения
- Функция в математике – некоторое соответствие элементов одного множества элементам другого множества, определённое таким образом, чтобы элементу первого множества соответствовал только один элемент другого множества

В функциональном программировании используется понятие функций именно в математическом варианте

Функциональное программирование в Java



**Java – не функциональный язык
программирования. Однако он даёт
некоторые возможности написать
код в функциональном стиле**

Функциональное программирование в Java

Нет использования состояния

Возможность определять final-переменные и immutable-объекты:

```
public class ImmutableExample {  
    private final String paramString;  
    private final int paramInt;  
  
    public ImmutableExample(String paramString, int paramInt) {  
        this.paramString = paramString;  
        this.paramInt = paramInt;  
    }  
}
```

Функциональное программирование в Java

Детерминированность функций

Java предлагает наличие функциональных интерфейсов и лямбда-выражений для написания детерминированных функций:

```
@FunctionalInterface
public interface MyFunction {
    int calculate(int param1, int param2);
}
```

```
MyFunction myFunction = (a, b) -> a + b;
```

Функциональное программирование в Java

Монады

Java предлагает Stream API для реализации монад. Можно выполнять большую операцию как последовательность более мелких без использования побочных эффектов и хранения промежуточных результатов:

```
public List<Integer> multiplyListWithFilter(Collection<Integer> collection) {  
    return collection.stream()  
        .filter(e -> e % 2 == 0)  
        .map(e -> e * 2)  
        .collect(Collectors.toList());  
}
```


Функциональное программирование в Java

Функции высших порядков

Чтобы реализовать функций высших порядков, введён механизм ссылок на методы:

```
new HigherFunction() {  
    @Override  
    public Function<Integer, Integer> calculate(Function<Integer, Integer> function, Function<Integer, Integer> extraFunction) {  
        return value -> extraFunction.apply(function.apply(value));  
    }  
};
```

```
public static void main(String[] args) {  
    calculateList(Arrays.asList(1, 2, 3, 4, 5), a -> a * a);  
}  
  
public static List<Integer> calculateList(List<Integer> source, MyFunction function) {  
    return source.stream()  
        .map(function::calculate)  
        .collect(Collectors.toList());  
}
```

Лямбда-выражения



Функциональный интерфейс в Java

Интерфейс, который содержит только один абстрактный метод. Основное назначение — использование в лямбда-выражениях и ссылках на методы



Функциональный интерфейс в Java

@FunctionalInterface

```
interface MyFunctionalInterface<T> {  
    boolean test(T value);  
}
```

Функциональный интерфейс **MyFunctionalInterface** имеет всего один метод без реализации **test()**, который принимает один аргумент типа **T**. Результатом выполнения метода **test()** является **boolean**.

Лямбда-выражения

Набор инструкций, которые можно:

- сохранить в ссылочную переменную
- передать в любой метод в качестве аргумента
- вызвать в любой момент
- исполнить один или несколько раз



Лямбда-выражения в Java

Упрощённая запись анонимного класса, реализующего функциональный интерфейс

() -> λ

Синтаксис

Основу лямбда-выражения составляет **лямбда-оператор**, который обозначается символом **«->»**.

Этот оператор разделяет лямбда-выражение **на две части**:

- левая часть — параметры выражения
- правая — тело лямбда-выражения, где выполняется действие

Синтаксис:

```
(параметры) -> {тело метода}
```

Структура

- Имеют от 0 и более входных параметров
- Параметры указываются либо явно, либо могут быть получены из контекста: `(int a)` можно записать как `(a)`
- Параметры заключаются в круглые скобки и разделяются запятыми: `(a, b)` или `(int a, int b)` или `(String a, int b, float c)`
- Без параметров необходимо использовать пустые круглые скобки: `() -> 42`
- Для одного параметра без указания типа скобки можно опустить: `a -> return a*a`
- Тело может содержать от 0 и более операторов
- Если тело состоит из одного оператора, его можно не заключать в фигурные скобки, а возвращаемое значение можно указывать без ключевого слова `return`

Примеры лямбда-выражений

```
(int a, int b) -> { return a + b; }
```

```
() -> System.out.println("Hello World");
```

```
(String s) -> { System.out.println(s); }
```

```
() -> 42
```

```
() -> { return 3.1415 }
```

Блоки кода в лямбда-выражениях

Существуют два типа лямбда-выражений:

- однострочное выражение
- блок кода

Пример однострочного лямбда-выражения

Пример однострочного выражения:

Блочные выражения обрамляются фигурными скобками. Здесь можно использовать вложенные блоки, циклы, конструкции if, switch, создавать переменные и т. д.:

Если блочное лямбда-выражение должно возвращать значение, то явным образом применяется оператор **return**:

```
s -> System.out.println(s);
```

```
s -> {  
    if (s.isEmpty()) {  
        s = "Hello World";  
    }  
    System.out.println(s);  
};
```

```
s -> {  
    boolean b = s.isEmpty();  
    return b;  
}
```


Отложенное выполнение

Одним из ключевых моментов в использовании лямбда-выражений является отложенное выполнение. Иными словами, определённое в одном месте программы лямбда-выражение можем вызвать при необходимости неопределённое количество раз в различных частях программы

Применение отложенного выполнения

- Код выполняется в отдельном потоке
- Один и тот же код выполняется несколько раз
- Код выполняется в результате какого-то события
- Код выполняется только в том случае, когда он действительно необходим

Как это работает

Лямбда-выражение не выполняется само по себе, а образует реализацию метода, определённого в функциональном интерфейсе.

Чтобы объявить и использовать лямбда-выражение, необходимо пройти ряд этапов:

- 1 определить ссылки на функциональный интерфейс:

```
Comparator<String> comparator;
```

- 2 создать лямбда-выражения:

```
comparator = (a, b) -> b.compareTo(a);
```

- 3 использовать лямбда-выражения в виде вызова метода интерфейса:

```
Collections.sort(cities, comparator);
```

Обратите внимание

- Параметры лямбда-выражения соответствуют параметрам единственного метода `compare()` интерфейса `Comparator`, а результат соответствует возвращаемому результату метода интерфейса:

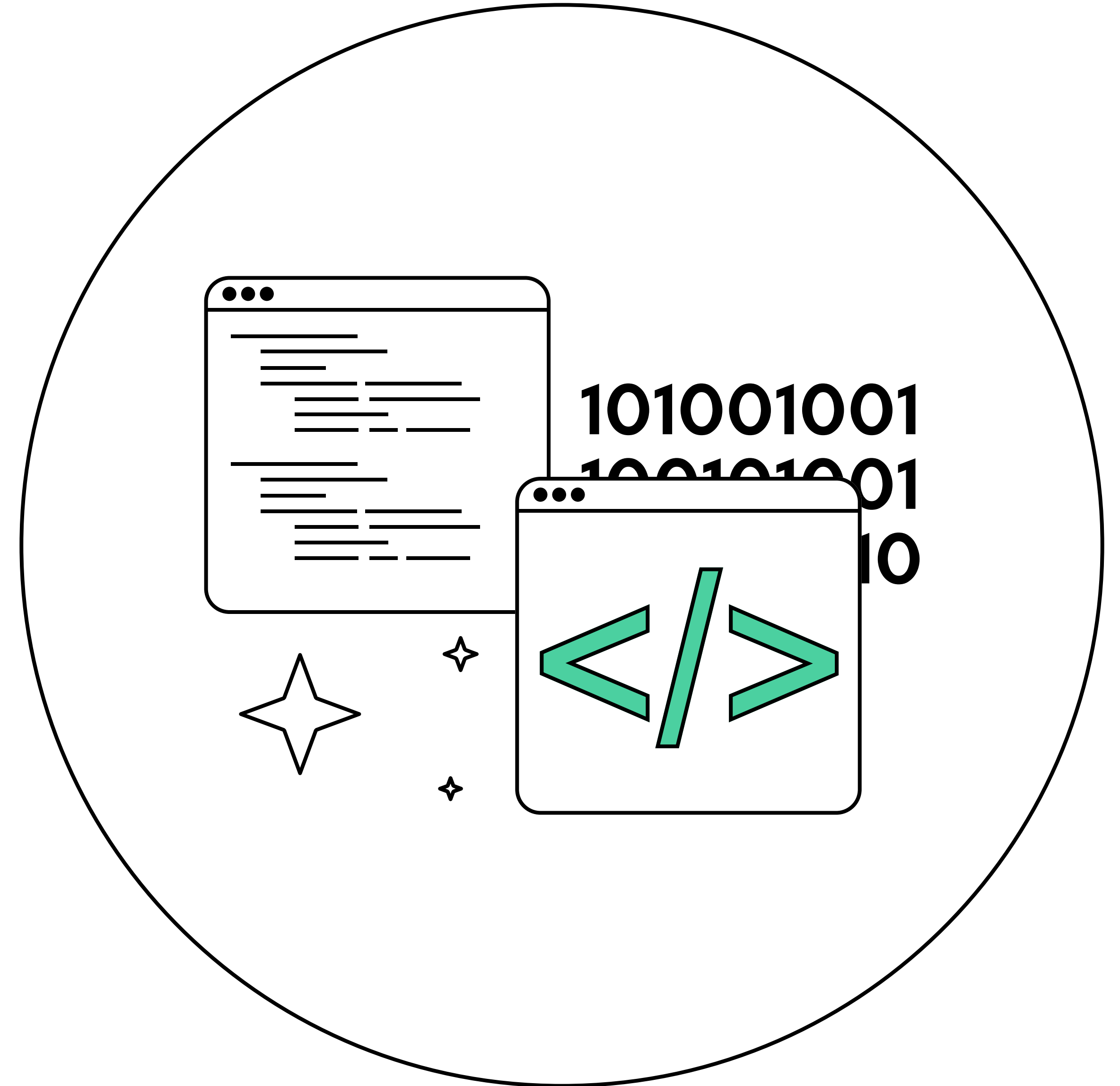
```
@FunctionalInterface  
public interface Comparator<T> {  
    int compare(T var1, T var2);  
}
```

- В качестве тела метода `compare()` интерфейса `Comparator` нами реализован вызов у второго параметра `b` типа `String` метода `compareTo()`, который сравнивает объекты `a` и `b` и возвращает `int`:

```
b.compareTo(a)
```

Лямбда-выражение как аргумент

Лямбда-выражение представляет набор инструкций, которые можно выделить в отдельную переменную и затем многократно вызвать в различных местах программы



Лямбда-выражение как аргумент

Пример: определим лямбда-выражение в качестве переменной* и передадим её как аргумент функции*.*:

```
public static void main(String[] args) {  
    List<String> cities = Arrays.asList("Мадрид", "Париж", "Москва", "Токио");  
  
    Comparator<String> comparator1 = (a, b) -> b.compareTo(a);  
    Comparator<String> comparator2 = (a, b) -> a.compareTo(b);  
  
    sortList(cities, comparator2);  
}  
  
public static <T> void sortList(List<T> list, Comparator<T> comparator) {  
    list.sort(comparator);  
    list.forEach(System.out::println);  
}
```

*Переменная — это поименованная либо адресуемая иным способом область памяти, адрес которой можно использовать для осуществления доступа к данным. **Аргумент функции — это переменная, которая передаётся методу при его вызове

Встроенные функциональные интерфейсы



Встроенные функциональные интерфейсы

В JDK 8 было добавлено несколько встроенных функциональных интерфейсов, широко применяемых в Stream API.

- Predicate<T>
- Consumer<T>
- Function<T,R>
- Supplier<T>
- UnaryOperator<T>
- BinaryOperator<T>

Predicate

Функциональный интерфейс `Predicate<T>` проверяет соблюдение некоторого условия. Если оно соблюдается, то возвращается значение `true`. В качестве параметра лямбда-выражение принимает объект типа `T`:

```
public interface Predicate<T> {  
    boolean test(T t);  
}
```

Пример:

```
Predicate<Integer> isPositive = x -> x >= 0;  
System.out.println(isPositive.test(5)); // true
```

Function

Функциональный интерфейс `Function<T,R>` представляет функцию перехода от объекта типа `T` к объекту типа `R`:

```
public interface Function<T, R> {  
    R apply(T t);  
}
```

Пример:

```
Function<Integer, String> convert = x -> x + " долларов";  
System.out.println(convert.apply(5)); // 5 долларов
```


Consumer

Consumer<T> выполняет некоторое действие над объектом типа **T**, при этом ничего не возвращая:

```
public interface Consumer<T> {  
    void accept(T t);  
}
```

Пример:

```
Consumer<Integer> printer = x -> System.out.printf("%d долларов \n", x);  
printer.accept(600); // 600 долларов
```

Supplier

`Supplier<T>` не принимает никаких аргументов, но возвращает объект типа `T`:

```
public interface Supplier<T> {  
    T get();  
}
```

Пример:

```
Supplier<String> stringFactory = () -> "new";  
String str = stringFactory.get();  
System.out.println(str); // new
```

UnaryOperator

UnaryOperator<T> принимает в качестве параметра объект типа **T**, выполняет над ними операции и возвращает результат операций в виде объекта типа **T**:

```
public interface UnaryOperator<T> {  
    T apply(T t);  
}
```

Пример:

```
UnaryOperator<Integer> square = x -> x * x;  
System.out.println(square.apply(5)); // 25
```


Supplier

`Supplier<T>` не принимает никаких аргументов, но возвращает объект типа `T`:

```
public interface Supplier<T> {  
    T get();  
}
```

Пример:

```
Supplier<String> stringFactory = () -> "new";  
String str = stringFactory.get();  
System.out.println(str); // new
```

BinaryOperator

BinaryOperator<T> принимает в качестве параметра два объекта типа **T**, выполняет над ними бинарную операцию и возвращает результат в виде объекта типа **T**:

```
public interface BinaryOperator<T> {  
    T apply(T t1, T t2);  
}
```

Пример:

```
BinaryOperator<Integer> multiply = (x, y) -> x * y;  
System.out.println(multiply.apply(3, 5)); // 15
```

Свой функциональный интерфейс

Если ни один из встроенных интерфейсов не подходит для решения поставленной задачи, то можно создать собственный функциональный интерфейс.

Зачастую, это используется при создании так называемых **колбэков** — объектов, которые определяются в классе **A** и передаются в качестве аргумента в класс **Б**:

```
@FunctionalInterface
public interface OnCompleteListener<T> {
    void onDone(T v);
}
```

Если необходимо вернуть результат работы класса **Б** в класс **A**, то у переданного ему **колбэка** необходимо вызвать метод функционального интерфейса

Ссылки на метод



Ссылки на метод

В Java 8 добавилась новая функциональность, названная **ссылкой на метод**.

Если лямбда-выражение используется для вызова метода **функционального интерфейса**, можно его заменить ссылкой на метод. Это компактная и простая форма лямбда-выражения.

Есть несколько типов ссылок на метод:

- ссылка на статический метод
- ссылка на метод экземпляра класса
- ссылка на конструктор

Ссылка на статический метод

Можно ссылаться на статический метод класса.

Синтаксис:

```
ContainingClass::staticMethodName
```

Создадим статический метод в классе `Main` и сделаем на него ссылку при реализации лямбда-выражения:

```
public static void main(String[] args) {  
    Consumer<String> sayable = Main::saySomething;  
    sayable.accept("Hello, this is static method");  
}  
  
private static void saySomething(String s) {  
    System.out.println(s);  
}
```

В роли функционального интерфейса выступает интерфейс `Consumer` с одним методом без реализации `accept()`. В роли статического метода выступает метод `saySomething()`, который выводит в консоль строку

Ссылка на метод объекта

Можно ссылаться на метод экземпляра класса.

Синтаксис:

```
containingObject::instanceMethodName
```

Создадим метод в классе **Main** и сделаем на него ссылку при реализации лямбда-выражения:

```
public static void main(String[] args) {  
    Main main = new Main();  
    Consumer<String> sayable = main::saySomething;  
    sayable.accept("Hello, this is non-static method");  
}  
  
private void saySomething(String s) {  
    System.out.println(s);  
}
```

В роли функционального интерфейса выступает интерфейс **Consumer** с одним методом без реализации **accept()**. В роли метода выступает метод экземпляра класса **saySomething()**, который выводит в консоль строку

Ссылка на конструктор класса

Можно ссылаться на конструктор класса.

Синтаксис:

```
ClassName::new
```

Определим функциональный интерфейс **Consumer** и класс **Main** с конструктором:

```
public Main(String string) {  
    System.out.println(string);  
}  
  
public static void main(String[] args) {  
    Consumer<String> sayable = Main::new;  
    sayable.accept("Hello, this is Main class");  
}
```

В роли функционального интерфейса выступает интерфейс **Consume** с одним методом без реализации **accept()**. В качестве класса выступает **Main**, в конструктор которого передаётся строка и выводится в консоль